SDVworld-index

Engineering manual — everything needed to take the project over.
Version 1.1 · `samanamarasinghe/SDVworld-index`

An index of the Synthetic Data Vault ecosystem: papers, preprints, theses, code repositories, documentation, tutorials, case studies and blog posts that use, extend, evaluate or merely cite SDV. It is a static site on GitHub Pages, built from hand-curated JSON shards by one Python script, with no backend and no database.

This manual has four parts: how to **use** the site, how it is **built**, what **data** it holds and what is missing from it, and what is **left to do** — separating the mechanical work from the work that needs a human to make a judgement call.

> **The four rules that matter most.** Breaking one is expensive.
>
> - Shards are **append-only**. A correction is a new record that sorts after the one it corrects, matched by `id`.
> - **Never hand-edit** `data/sdv-index.json`, `data/build-info.json` or anything under `data/site/`. They are generated; CI regenerates the first two.
> - **Read the source before writing a summary.** Citing a paper is not using the software.
> - **Verify every push with a byte diff.** Two silent corruptions have got through review.

1 · User guide

The site is one page: `samanamarasinghe.github.io/SDVworld-index/`. Filters are at the top, results below. Everything is client-side — no login, no server, nothing is recorded about what you search.

The two sliders

SDV importance is the one to reach for first. It asks how central SDV is to the entry, on a curator-assigned 0–6 scale, and it is the difference between browsing the field and browsing everything that ever mentioned SDV.

Level	Meaning	Entries
6	First-party — produced by the SDV project itself	81
5	SDV *is* the work: a fork or reimplementation	73
4	Load-bearing — remove SDV and the work does not stand	1,389
3	One of several baselines	1,626
2	Method described, not run	577
1	Passing citation	957
0	Everything, including 215 unrated entries and the 44-row uncurated pool	4,962

The default is **1 or above**, which shows 4,703 entries. Note the asymmetry: dropping to 0 adds only 259 entries, while moving to 4 removes more than three thousand.

Popularity is attention drawn, and *nothing else* — GitHub stars, forks, contributors and commits for code; citations for papers; log-compressed onto one 0–1 scale so the two are comparable. It is a percentile cut over everything currently in view. An entry with no attention signal at all sits at a neutral 0.3 rather than at zero.

Popularity and importance are deliberately independent: work can be widely used but peripheral to SDV, or obscure and built entirely on it.

Search

By default the box matches **title, summary and author**. Each word is matched separately and all of them must appear, in any order; the last word matches as a prefix, so results narrow as you type. Accents are folded, so `muller` finds *Müller* and `naive` finds *naïve*.

Untick **also search summaries** to match titles only — narrower, and useful when a common word is swamping the results. There are no wildcards.

> Author names are indexed, but the index arrives a moment after the page does. A search typed in the very first instant matches title and summary only, and re-runs itself once the author index lands.

Facets

Checkbox facets are **OR within, AND across**: ticking two kinds shows either, but ticking a kind and a sector shows only entries with both. Each count is *self-excluding* — the number beside a value is how many entries you would get if you ticked it, given everything else already selected, which is why a value can read 0 without vanishing.

Sector and **Affiliation** carry button groups that work the opposite way: they *permit* rather than select. All buttons lit filters nothing; unlighting *Americas* removes every entry with any American organization, even if it also has a European one. That is what makes a single lit button read as "only here".

Not specified is a real value, not an absence — the curator looked and found nothing to record. Authors and organizations have no such value, because a missing author list is a gap in the data rather than a statement about the work.

Results

The first 100 are drawn, with **Show 100 more** and **Show all** beneath. "Show all" is honest about being slow: rendering all 4,703 takes about 1.2 seconds and builds 117,000 elements. Grouping headings read *visible of total*, and an entry with two use cases genuinely appears under both, so section totals can exceed the entry count.

Summary expands the curator's note, which always ends by saying which part of SDV is involved and how. **Show open questions** reveals `needs` — the honest record of where the index is soft.

2 · Architecture

Three stages, one direction, no backend: curated shards are merged by a Python build into two independent outputs — a public export nobody's browser reads, and a projection built solely for the page.

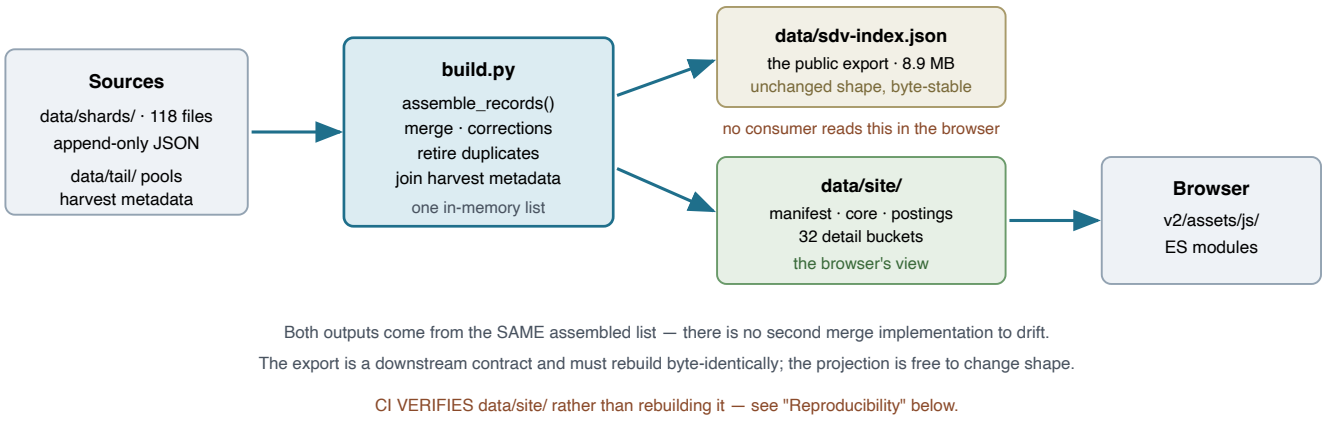

Figure 1 — the build. One merge, two outputs, one direction.

What the browser downloads

File	When	gzip	Holds
`manifest.json`	eager	0.4 KB	counts, file sizes, `data_hash` (the cache identity)
`core.json`	eager	996 KB	every field the filter, the sort and a collapsed card need
`summary-postings.json`	eager	343 KB	15,143-token inverted index over title + summary
`author-postings.json`	after paint	100 KB	12,420-token index over author names
`detail/00–1f.json`	on demand	33 KB ea.	summary and needs only, 32 buckets

1.38 MB eager, against a design budget of 1.50 MB. The previous version loaded 2.59 MB, of which 0.55 MB was two raw harvest pools downloaded so the browser could discover the 44 rows of them that survive de-duplication — work now done at build time.

The record, and what does not travel

The projection is **deliberately lossy**. `core.json` drops `source_channel`, `evidence_tier`, `openalex_id`, `countries`, the raw aligned affiliation lists, and the popularity inputs now folded into a single score. Those stay in the public export for downstream consumers. It *adds* five precomputed values the page would otherwise derive on every keystroke:

Field	Derived from	Why precomputed
`organizations`	`affiliations` split on `;`	one element can hold several institutions
`aff_type`	`affiliation_types`	academic / non-academic / unaffiliated, and they overlap
`aff_region`	`affiliation_countries`	every region enumerated; an unplaceable country gets none

`pop`	stars, forks, contributors, commits, citations	log-compressed blend; see the warning below
`b`	`sha256(id)[0] & 0x1f`	which detail bucket holds the summary

The filter engine

The previous version walked the corpus **thirteen times per interaction** — once for the list, once for the header count, and once per facet for its self-excluding count — re-sorting the whole corpus inside each walk to find the popularity percentile. A single click cost between 0.7 and 3.2 seconds.

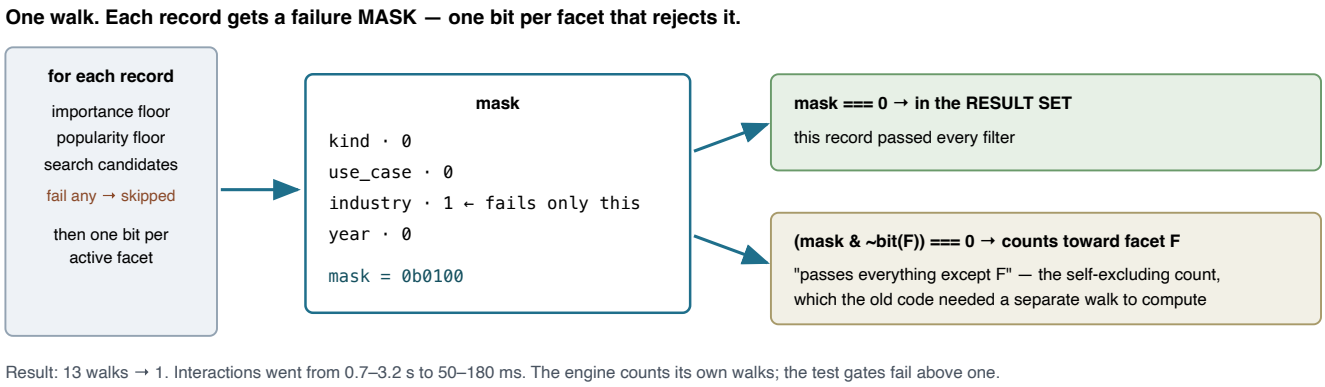

Figure 2 — why one pass is enough for both the results and every facet count.

Runtime modules

Native ES modules under `v2/assets/js/`; no bundler, no build step for the front end.

Module	Responsibility
`vocab.js`	Labels, controlled vocabularies, region tables. **Extracted verbatim from the old runtime by script**, not retyped; `scripts/check_vocab_parity.py` fails on drift.
`data.js`	Loads the projection; normalizes each record once; the `Details` bucket loader.
`search.js`	Tokenizing, folding, postings, the multi-index `Index`.
`engine.js`	The one-pass mask walk. Caches its result on a state signature.
`order.js`	Sorting and grouping, including the page-limit group plan.
`render.js`	Cards, the 100-record page, lazy summary and needs, BibTeX.
`state.js`	State, the facet panel, input coalescing.
`app.js`	Wiring. Exported as a class so harnesses can drive it.

Three traps, each of which has already bitten.

- **Popularity is not rounded, on purpose.** `math.log1p` is not correctly rounded and libm differs by platform, so this corpus built on macOS and on CI's Linux differs in the last bit for 395 records. Rounding to 9 dp looks safe — it collapses only 10 pairs that should have scored identically — but those pairs are *ordered* in the old runtime, the golden corpus recorded that order, and quantizing failed 76 golden states. Fidelity wins. CI therefore **verifies** `data/site/` instead of rebuilding it, so the bytes served are always the bytes that were tested.
- **Paths resolve through `import.meta.url`.** Pages serves under `/SDVworld-index/`, so a root-relative data path works locally and 404s in production.

- **The build tokenizes in Python, the runtime in JavaScript.** If they diverge, a query is looked up under a key the text was never filed under and search silently returns too little. Both are pinned to a shared table, `tests/semantic/tokenizer-cases.json`, checked from each side.

3 · The data

4,918 curated entries, assembled from 118 append-only shards, plus 44 uncurated pool survivors visible only at importance 0. Publication years span 2011–2026.

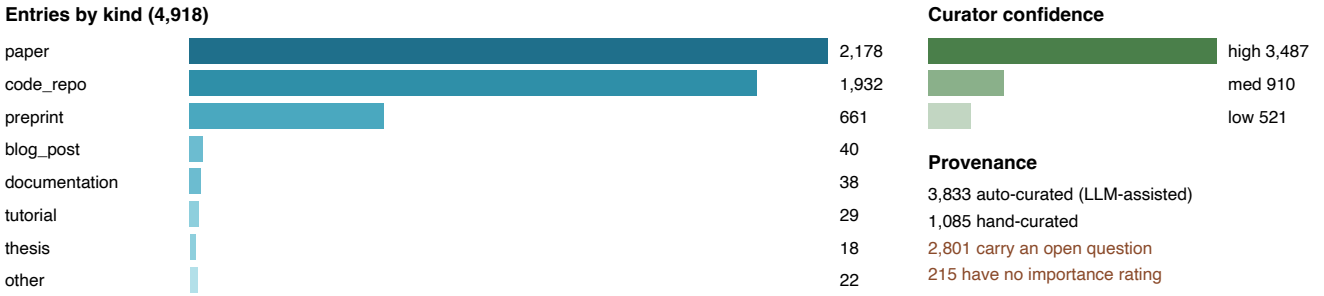

Figure 3 — composition of the curated index.

What was gathered

Records come from OpenAlex (citation graph around the SDV papers), a GitHub code search for SDV import patterns, and hand-curation of first-party and notable sources. Bibliographic and repository metadata — year, DOI, venue, stars, forks, contributors, commits — are *joined at build time* from `data/tail/`; a curator's value always wins over the join, and the join only ever fills a field a shard left empty.

Author affiliations are the most laborious part of the dataset: 10,599 distinct author names resolve to 2,655 distinct organizations, each classified academic or non-academic and placed in a country, through `data/affiliation-normalizations.json` and a stack of curated override files.

What is missing — and how much

Gap	Entries	Consequence
No author list	1,364	28% invisible to author search and the Author facet
No affiliation	1,364	same records; they read *Affiliation not found*
No importance rating	215	hidden at the default floor of 1 — they only appear at 0
Open question recorded	2,801	a verification the curator could not complete
Low confidence	521	the summary may not reflect the source
No DOI	2,100	no stable pointer; the URL may rot
No year	13	sorts last, grouped under *Undated*

> **The largest single gap is affiliation coverage.** 1,364 of 4,918 entries — nearly three in ten — carry no author or affiliation at all, which means every sector, region and organization figure on the site is computed over 72% of the index. The counts are correct for what is recorded; they are not a census.

Two further caveats worth knowing before trusting a number:

- **2,801 entries carry an open question**, not the 114 the old interface's tooltip claimed. That tooltip is stale and is listed as a to-do below.

- **3,833 of 4,918 summaries are LLM-assisted** and carry a `reviewed` flag. Confidence is the curator's own estimate of whether the source was actually read, and 521 say *low*.

4 · Working on it

The commands

`python3 build.py`	merge shards, report counts, write nothing
`python3 build.py --write`	write the export, the stamp and the projection
`python3 tests/validate.py`	schema, vocabulary, pointer and alignment checks
`python3 tests/build_tests.py`	15 checks on the projection
`python3 tests/gates.py --stage 2b`	the eleven structural gates
`python3 scripts/verify_site.py`	is the committed projection current?
`python3 scripts/serve.py`	localhost:8765, with a POST sink for the browser harnesses

The test suite, and what each part can actually catch

They are layered deliberately: each catches a class the one above cannot see.

Suite	Catches	Blind to
Golden differential `tests/oracle/` 293 frozen states	any change in which records match, in what order, with what facet counts	everything about the DOM — it drives the engine directly
Semantic suite `tests/semantic/` 36 hand-authored cases	whether the semantics are *right*, not merely unchanged	anything not written down as a case
Build tests `tests/build_tests.py`	projection integrity, export byte-identity, tokenizer agreement	runtime behaviour
UI parity `tests/parity/`	every visible detail of both real pages, driven by real clicks	the old page will not exist forever — see below
Gates `tests/gates.py`	the structural budgets; fails on missing or stale evidence	wall-clock timing, which is recorded and never gated

> **The parity harness has a shelf life.** It compares the current page against the previous one at `/v1/`, which the design keeps for one release only. When `/v1/` is removed, that harness stops being meaningful and the golden corpus becomes the only record of the old behaviour. Decide deliberately whether to freeze a final parity run before deleting it.

To-do — mechanical

Work that needs doing but not deciding.

- **Retire `/v1/`** after one release, with the parity question above settled first.
- **The stale tooltip.** The open-questions button claims 114 entries carry one; the real figure is 2,801. The text is in `index.html`.

- **The BibTeX download path is not tested end to end.** Creation and revocation of the object URL are covered; the browser actually saving the file is not, because the test stubs delivery to avoid writing files during runs.
- **Rendered summaries use `innerHTML`**, as the old page did, because curated summaries carry inline links. The text is curator-authored and travels with the generated index, but it is worth a deliberate look.
- **Node and Playwright are not installed** on the maintainer's machine, so the browser harnesses are driven manually rather than in CI. They were written with no driver-specific API precisely so a Playwright driver can load the same URLs.
- **215 entries have no importance rating** and are therefore invisible at the default floor. Rating them is mechanical once someone reads the source.

Needs human judgement

Do not let an agent decide these.

- recall **Should `GAN` match inside `ctgan`?** The old page matched substrings anywhere and returned 3,589; the current page matches whole words and returns 1,311. Someone searching GAN arguably wants the CTGAN papers. Restoring it is a five-line change to match the final term as a substring of the vocabulary — at the cost of `he` matching inside `the` again.
- data **The affiliation gap.** Closing it for 1,364 entries is weeks of work and the only way to make the sector and region figures a census rather than a sample. Whether that is worth it is a judgement about what the index is *for*.
- data **How much LLM-assisted curation is acceptable?** 78% of summaries are auto-curated. The `reviewed` flag and the confidence rating exist to make that visible, but the threshold for publishing an unreviewed summary is a policy nobody has written down.
- scope **The uncurated pools.** 44 rows survive de-duplication and appear only at importance 0. Either curate them or drop them; leaving them is a third choice that costs a permanent branch in the code.
- scope **Shareable query-string state** was deferred as optional. It is the most-requested kind of feature for an index like this, and it interacts with every filter.
- policy **Unresolved curation questions** live in `docs/open-questions.md`. They are there because they need the owner's ruling; do not invent a resolution.

Where to read next

`README.txt`	the project in prose, and the traps — fastest orientation
`AGENTS.md · CLAUDE.md`	how to work here; the operating rules
`docs/schema.md`	the record schema and controlled vocabularies — the definition, parsed by validate.py
`docs/agent-guide.md`	curation procedure, the append-only rule, access routes to full text
`docs/perf/`	the performance redesign: design v2, five stage briefs, four handoffs
`TODO.txt`	what is left, with the commands that regenerate every count in it

Generated from `docs/manual/manual.html` by `scripts/build_manual.sh`. Figures are inline SVG; edit the HTML and re-run.